

Experiment 2

Aim: To develop practical knowledge of SQL by performing complex queries involving joins, aggregate functions, grouping, ordering, and ranking operations for database analysis.

Code:

Problem 1: A university administration requires an analytical performance report for each academic department. Write SQL queries to:

1. Create the required tables (**Departments, Students, Subjects, and Marks**).
2. Insert suitable sample records into each table.
3. Display the **average marks obtained by students in each department**.
4. The output should contain the **Department Name** and **Average Marks**, sorted in descending order of average marks.

Solution:

-- Create Tables

```
CREATE TABLE Departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(50) NOT NULL,  
    dept_id INT REFERENCES Departments(dept_id)  
);
```

```
CREATE TABLE Subjects (  
    subject_id INT PRIMARY KEY,  
    subject_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Marks (  
    student_id INT REFERENCES Students(student_id),  
    subject_id INT REFERENCES Subjects(subject_id),  
    marks_scored INT NOT NULL,  
    PRIMARY KEY (student_id, subject_id)  
);
```

-- Insert Records

```
INSERT INTO Departments (dept_id, dept_name) VALUES  
(1, 'Computer Science'),  
(2, 'Mathematics');
```

```
INSERT INTO Students (student_id, student_name, dept_id) VALUES  
(101, 'Alex', 1),  
(102, 'Bella', 1),
```

```
(103, 'Charlie', 2);
```

```
INSERT INTO Subjects (subject_id, subject_name) VALUES  
(501, 'Data Structures'),  
(502, 'Calculus');
```

```
INSERT INTO Marks (student_id, subject_id, marks_scored) VALUES  
(101, 501, 85),  
(101, 502, 90),  
(102, 501, 75),  
(103, 502, 95);
```

```
-- Display Department-wise Average Marks  
SELECT  
    d.dept_name AS Department,  
    ROUND(AVG(m.marks_scored),2) AS "Average Marks"  
FROM Departments d  
JOIN Students s  
ON d.dept_id = s.dept_id  
JOIN Marks m  
ON s.student_id = m.student_id  
GROUP BY d.dept_name  
ORDER BY AVG(m.marks_scored) DESC;
```

Problem 2:

A multinational organization maintains employee and department information in separate normalized tables. Write SQL queries to perform the following tasks:

1. Create the **Department** and **Employee** tables.
2. Insert suitable sample records into both tables.
3. Display the employee(s) who receive the **highest salary in each department**. If multiple employees have the same highest salary within a department, display all of them.
4. Find the **second highest earning employee(s)** in the organization.
5. Display the results in an appropriate format.

Solution:

```
-- Create Department Table  
CREATE TABLE Department  
(  
    id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);  
  
-- Create Employee Table  
CREATE TABLE Employee  
(  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    salary INT,  
    department_id INT,  
    FOREIGN KEY (department_id)
```

```

REFERENCES Department(id)
);

-- Insert Records into Department
INSERT INTO Department
VALUES
(1,'IT'),
(2,'SALES');

-- Insert Records into Employee
INSERT INTO Employee
VALUES
(1,'JOE',70000,1),
(2,'JIM',90000,1),
(3,'HENRY',80000,2),
(4,'SAM',60000,2),
(5,'MAX',90000,1);

-- Display Highest Salary Employee(s) in Each Department
SELECT
    d.dept_name,
    e.name,
    e.salary
FROM Employee e
JOIN Department d
ON e.department_id = d.id
WHERE e.salary =
(
    SELECT MAX(salary)
    FROM Employee
    WHERE department_id = d.id
)
ORDER BY d.dept_name;

-- Display Second Highest Salary Employee(s)
SELECT *
FROM Employee
WHERE salary =
(
    SELECT MAX(salary)
    FROM Employee
    WHERE salary <
(
    SELECT MAX(salary)
    FROM Employee
)
)
);

```

Problem 3: Two legacy HR systems (A and B) maintain separate records of employee salaries. These records may overlap. Management wants to merge these datasets and identify each unique employee (by EmpID) along with their lowest recorded salary across both systems.

Objective

1. Combine the records from Table A and Table B.
2. Return each EmpID with their lowest salary and the corresponding Employee Name (Ename).

Solution:

```
SELECT
    EMPID,
    MIN(ENAME) AS ENAME,
    MIN(SALARY) AS SALARY
FROM
(
    SELECT * FROM A

    UNION ALL

    SELECT * FROM B
) AS COMBINED
GROUP BY EMPID;
```

Problem 4: Write an SQL query to display the **first name**, **last name**, **city**, and **state** of every person. If a person's address is not available, return **NULL** for the city and state.

Solution:

```
SELECT
    P.firstName,
    P.lastName,
    A.city,
    A.state
FROM Person AS P
LEFT JOIN Address AS A
ON P.personId = A.personId;
```

Learning Outcome:

1. Understand and implement advanced SQL queries for effective data retrieval.
2. Create and manage relational database tables using SQL DDL commands.
3. Insert, update, delete, and retrieve data using SQL DML statements.
4. Apply SQL joins to combine data from multiple related tables.